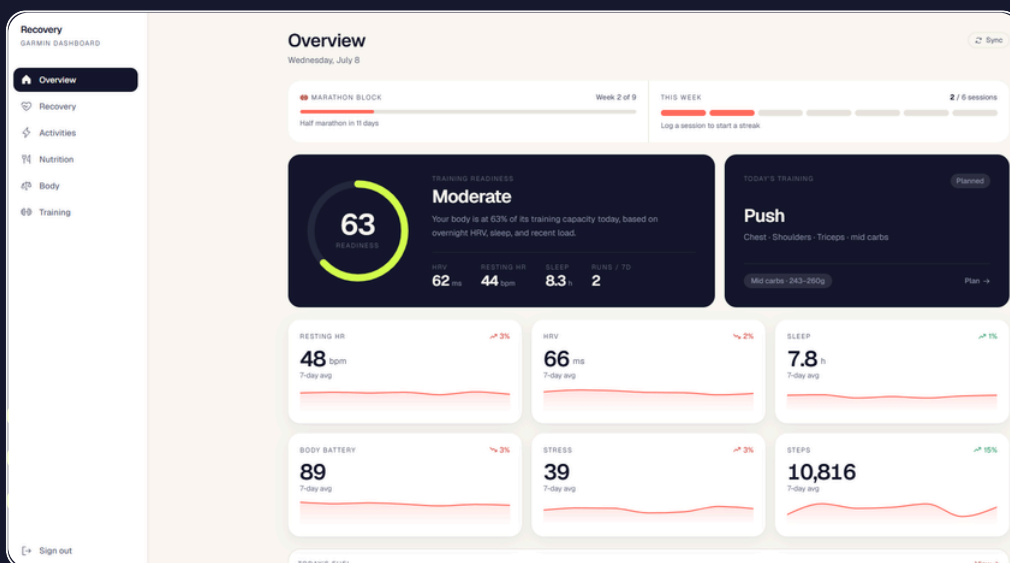


FIRST BUILD

Your first real AI tool, built this weekend.

No coding experience needed. Just you, Claude, and a working health dashboard with your real data in it by Sunday.



Building beats watching.

You won't learn AI from another YouTube video. You've watched plenty, but you're still here, with nothing built.

The people winning with AI aren't the ones who studied it. They're the ones who built with it. Claude changed the deal: you describe what you want in plain English, and it writes, runs and fixes the code. That means your job isn't to code, but to decide what gets built.

This pack gets you set up, then walks you through one real build: a health dashboard that pulls in your actual Garmin data. Something that can replace one or two of the subscriptions you already pay for. This document has the exact prompts, in order, plus fixes for when things wobble.

I'm not a developer. I'm an ex-consultant who got made redundant and bet on building things. Every project in here is something I built and use myself. The builds are the receipts.

The build&keep rules

- 1. Finish ugly:** A working v1 beats a perfect idea every time.
- 2. Make it yours:** These instructions and Claude's initial build are only the starting point. Change it, break it and personalise it so it's truly yours.
- 3. Breaking is building:** When something breaks, tell Claude. Fixing errors is part of the build, not a failure.

Setup: ten minutes, once.

Everything in this pack runs on the Claude desktop app. No coding background required. You type what you want built and Claude builds it. This page is a one-time setup, so once you're done, every build in the future starts in sixty seconds.

What you need

A computer running Windows or Mac, a Pro Claude subscription (about US\$20/month), and 10 minutes

0.1 DOWNLOAD THE CLAUDE DESKTOP APP

Go to the link below and download the app for your computer. Install it the same way you'd install anything else. Open the file, follow the prompts, done.

Download Claude

<https://claude.com/download>

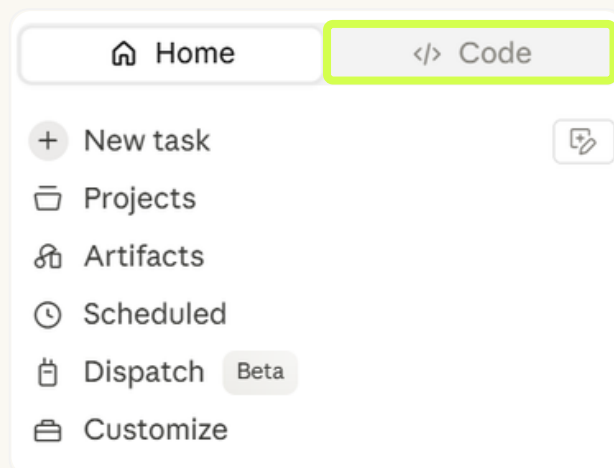
0.2 GET CLAUDE PRO

Claude Code requires a Pro subscription. If you don't have one yet:

1. Open the Claude app and sign in (or create a free account at claude.ai)
2. Click your profile icon → Upgrade to Pro
3. Choose the Pro plan (about US\$20/month)
4. Your subscription activates straight away

0.3 OPEN CLAUDE CODE

Once you're in the app, look at the top-left corner. You'll see two buttons:

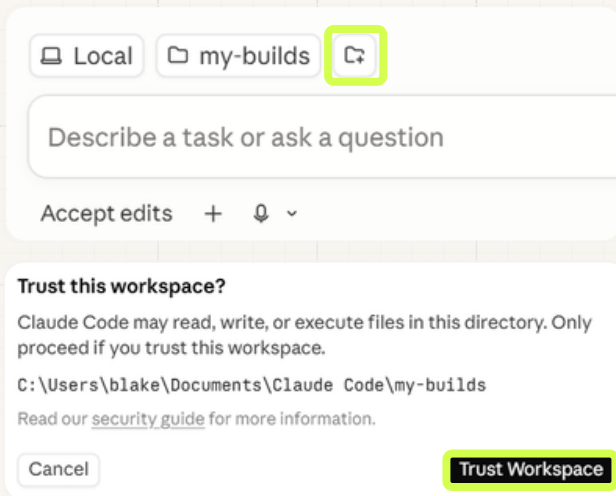


Open Claude Code

If the </> Code button is greyed out, your Pro subscription may not have activated yet. Restart the app, or check that your Pro subscription is active

0.4 CREATE A PROJECT FOLDER

Give Claude a place to put your work. Before you do anything else, create a new folder on your computer and call it 'my-builds' (or whatever you'd like!).



Click this to link a project folder

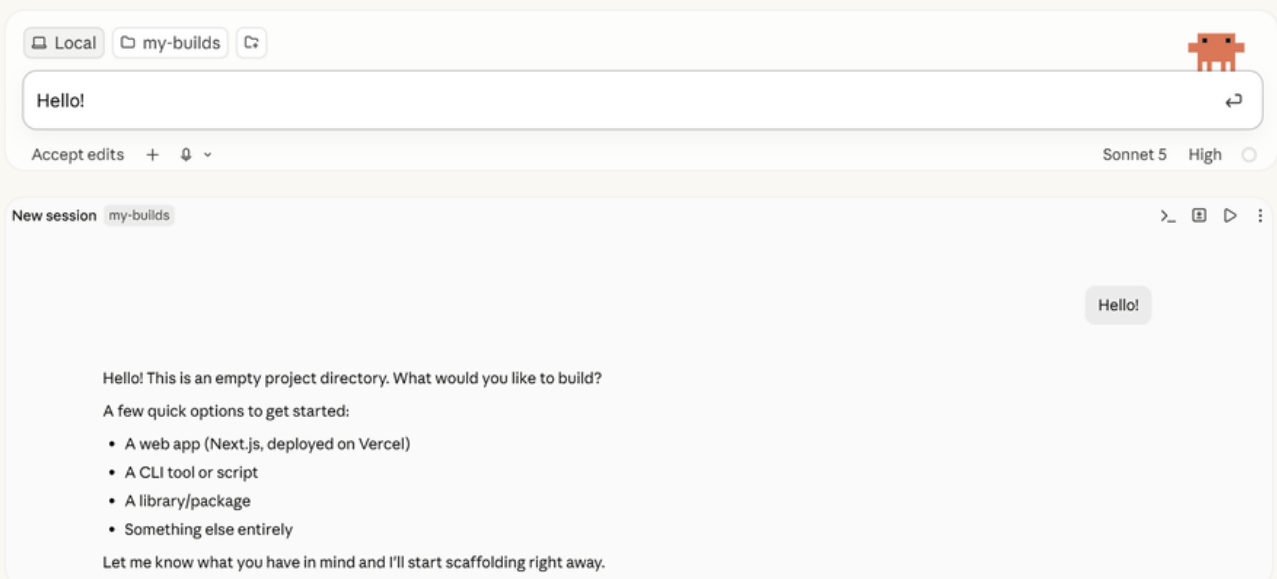
Claude Code needs access to a folder on your computer to put your work. Give each build its own folder. This keeps Claude from touching files it shouldn't.

Trust Workspace

Click 'Trust Workspace' once you've selected your chosen folder

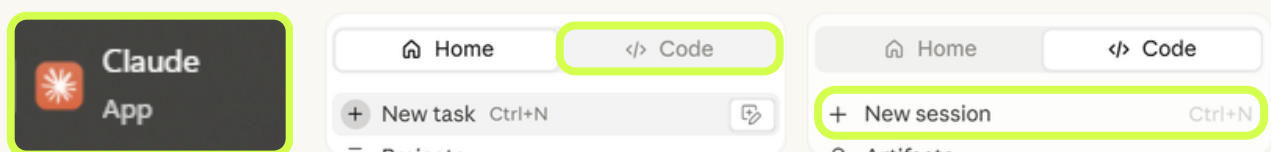
0.5 SAY HELLO

Type 'Hello' into the prompt bar, and hit enter:



Congratulations

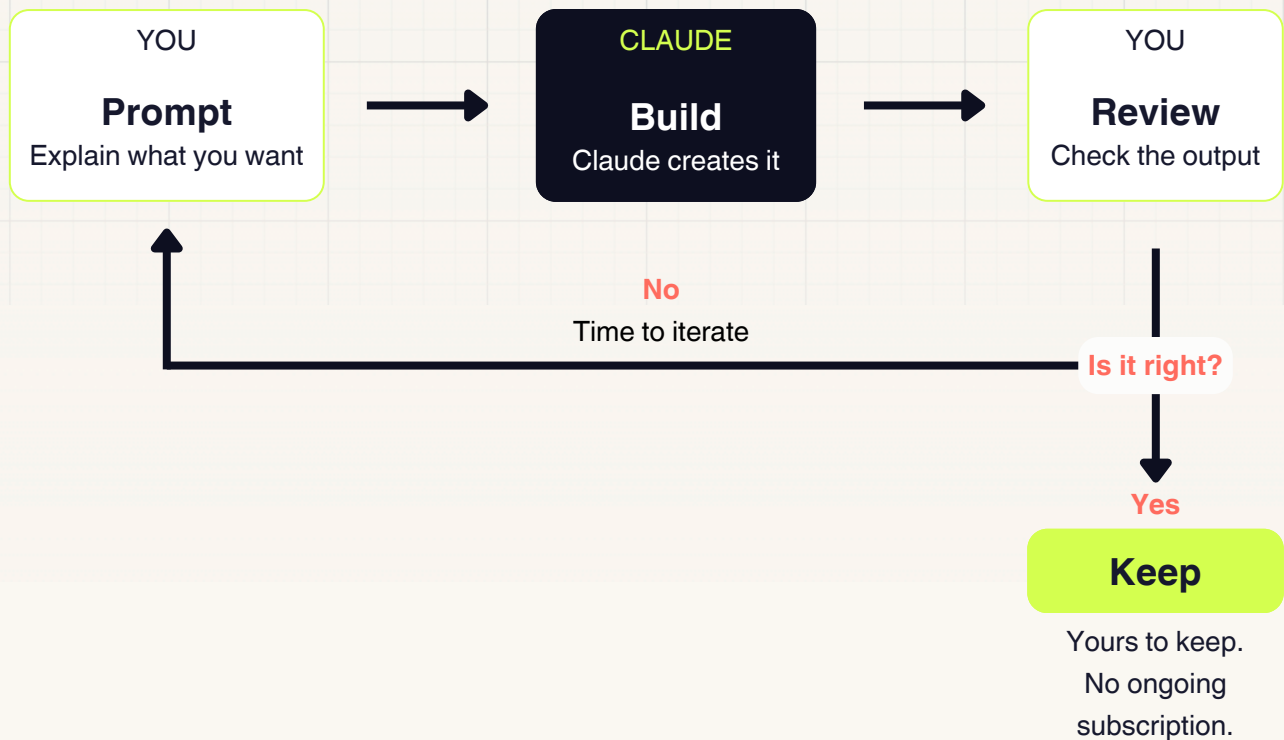
You just started your first chat in Claude Code. For future sessions, all you need to do is open the Claude desktop app, click `</> Code` in the top left corner, and start a new session.



The question now: what should we build first?

The fundamental loop.

Master one loop, and you can build anything. Every build runs on this fundamental loop, and so will everything you go on to build afterwards.



Why this works

Outcomes, not code

You describe what 'good' looks like. Claude handles the how. Your judgement and direction are the skill.

Everything is fixable

Everything that breaks is fixable. Describe and show Claude what has broken, so it can go and fix it for you.

One prompt at a time

Each prompt builds on the last. Ask for too much at once and the quality drops

Learn the loop, keep the builds

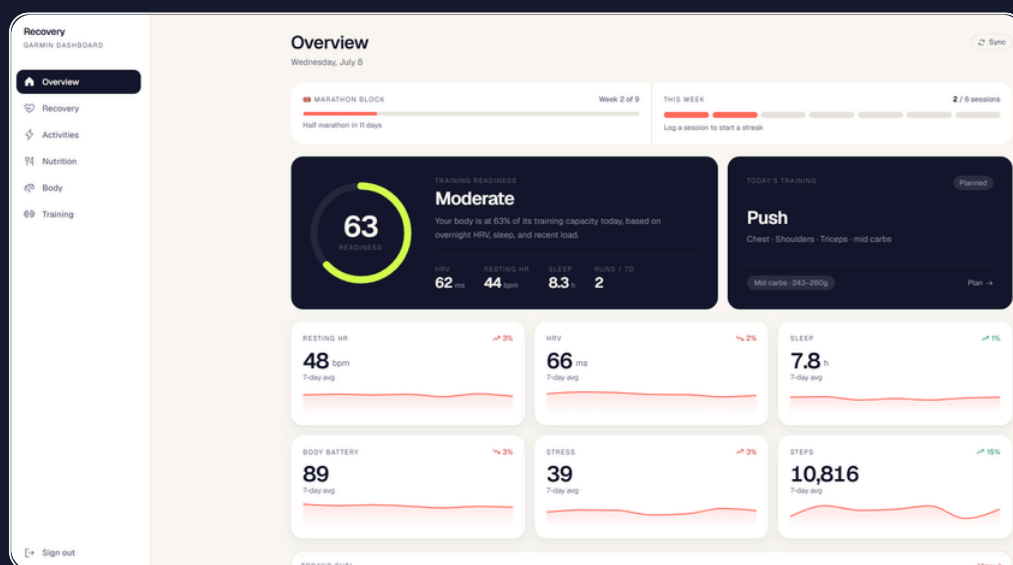
Get this and you're not following a tutorial anymore — you're building something of your own with your own skills, ideas and personality. Everything from here is just running the loop, one build at a time

YOUR FIRST BUILD

HEALTH DASHBOARD

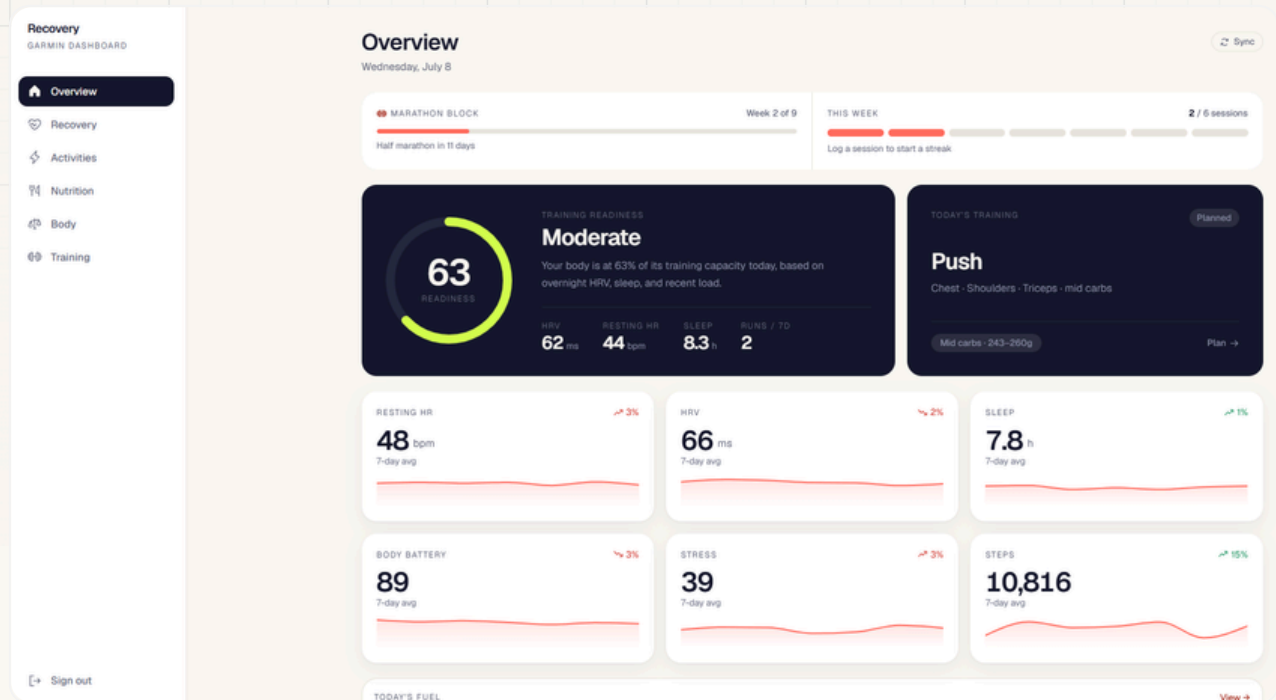
Elevate your Garmin data

Build a health dashboard that runs in your browser and uses your Garmin data, with room to take it further.



Health Dashboard

In three moves, you'll have your own Garmin data in a dashboard you built and keep. First, we put up the shell, then we sync your real numbers, and then we make it yours. Free, and yours to keep on your own computer.

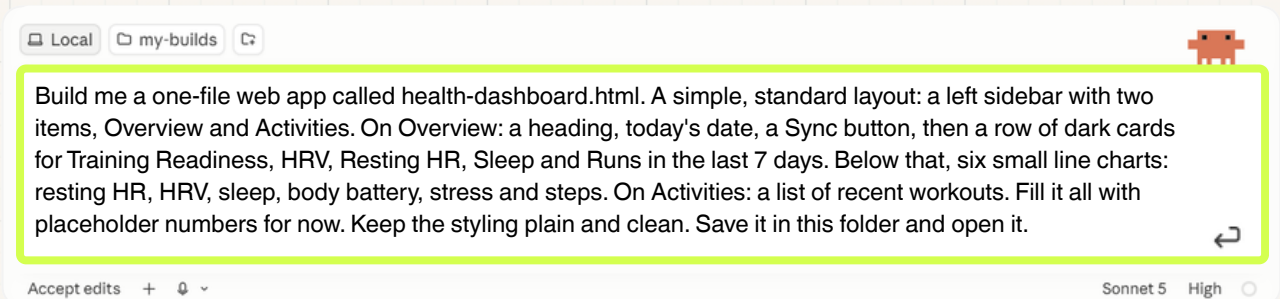


What you'll get

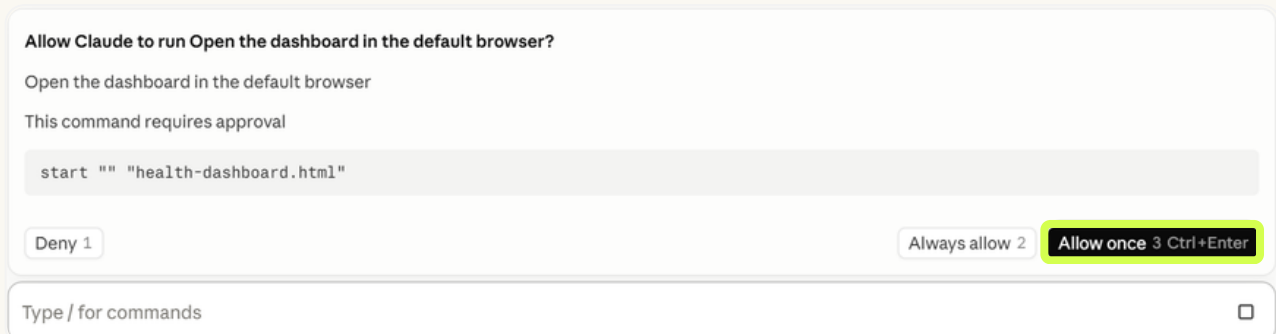
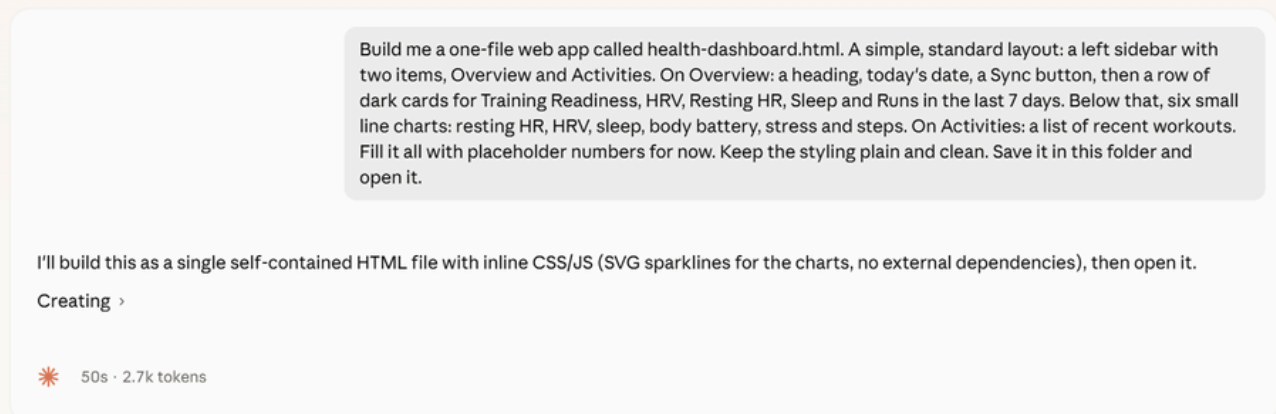
By the end of this build, you'll have synced your real Garmin data onto your device and be able to see it: your training readiness, HRV, resting HR, sleep and your activities, all in a dashboard you built and own.

Put up the shell

This first prompt builds the empty dashboard, with placeholders for all of the numbers that will be synced afterwards. A standard, clean layout that we'll keep plain on purpose (more on that below).



Paste this prompt into Claude Code, and press ↵



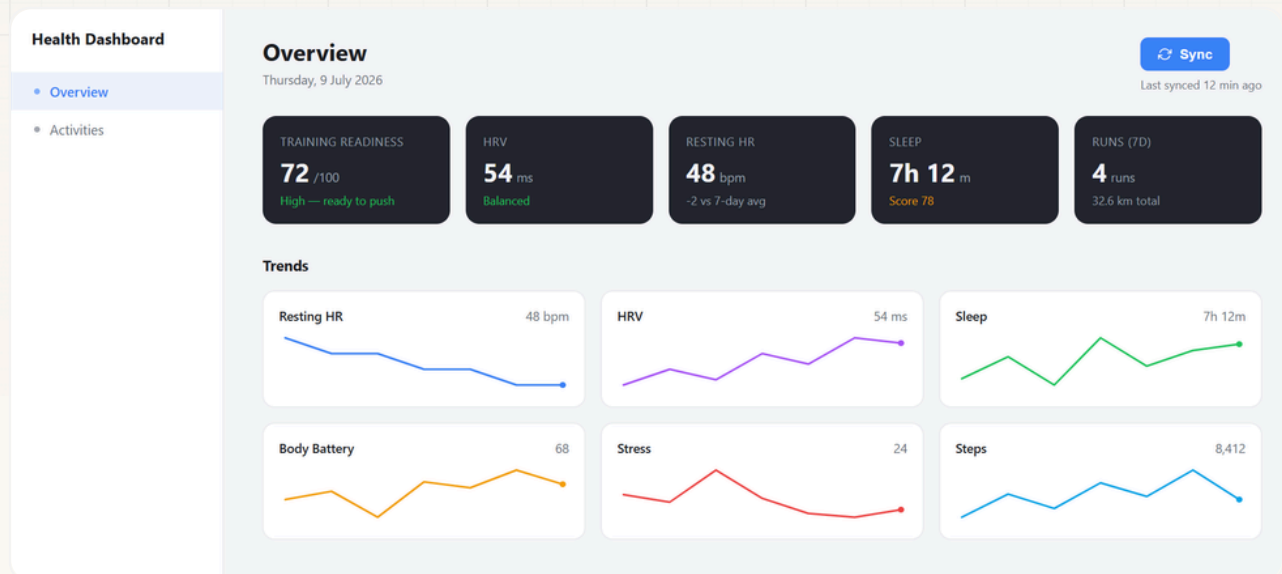
Claude will often ask for permission to complete actions. In this case, it should ask to open the dashboard it has built in your browser. Generally, clicking 'Allow once' is best.

See what you built

Claude should automatically open the dashboard in your browser. If not, ask Claude to open the dashboard or open it yourself by double-clicking 'health-dashboard.html' in your project folder.

All of the data in the dashboard is still placeholder. We'll add your Garmin data in the next step.

Claude builds with a little randomness baked in, so the very same prompt gives everyone a slightly different result. Yours won't match ours exactly, and that's completely normal.



Why it looks so plain, and 'AI slop'

AI can spit out an app in seconds that's generic, soulless, samey. The internet calls it 'AI slop.' We start plain and functional on purpose; as you keep building, you'll grow the taste to make something genuinely yours.

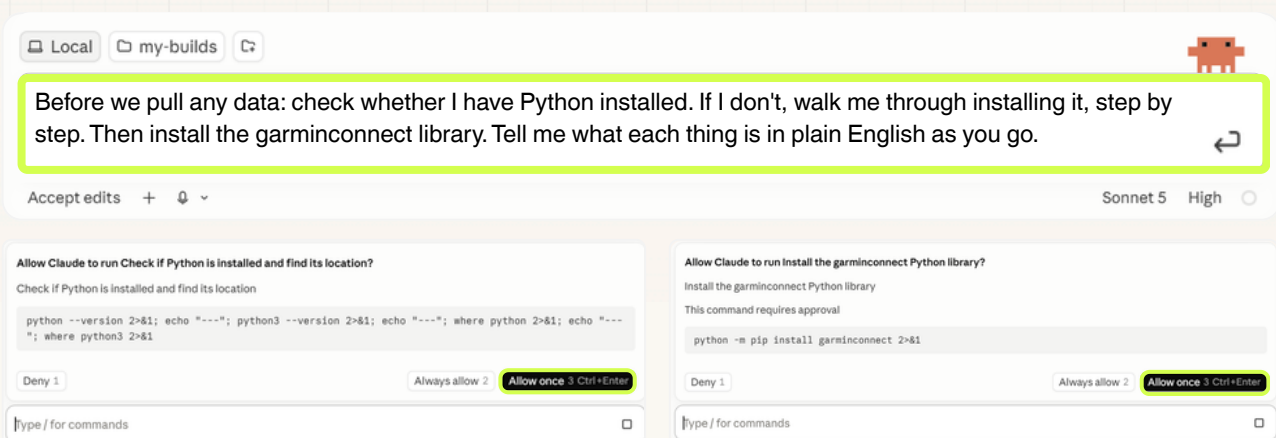
Sync your real Garmin data

Now it's time to make it yours, with your own Garmin data. We'll pull your data into the dashboard in four steps. It's important to do these in order, so don't skip ahead!

What's actually happening

Garmin has no public API, so we use an open-source tool called garminconnect to log in and pull your data down to a file on your own computer. This is read-only so it never changes anything on Garmin, and nothing leaves your machine.

2.1 GET PYTHON READY



The screenshot shows the Claude AI interface. At the top, there are tabs for 'Local' and 'my-builds'. A prompt is entered in the text area: 'Before we pull any data: check whether I have Python installed. If I don't, walk me through installing it, step by step. Then install the garminconnect library. Tell me what each thing is in plain English as you go.' Below the prompt, there are two permission dialogs. The first dialog asks 'Allow Claude to run Check if Python is installed and find its location?' and shows the command: `python --version 2>&1; echo "----"; python3 --version 2>&1; echo "----"; where python 2>&1; echo "----"; where python3 2>&1`. The second dialog asks 'Allow Claude to run Install the garminconnect Python library?' and shows the command: `python -m pip install garminconnect 2>&1`. Both dialogs have 'Deny', 'Always allow', and 'Allow once' (highlighted in yellow) buttons. The 'Allow once' button is labeled 'Allow once 3 Ctrl+Enter'.

Claude will ask for your permission to run a command or two to check if you've got Python and garminconnect installed, and install them if not. Click 'Allow once' when prompted.

Never installed anything like this?

That's fine, and it's a normal part of the process. Claude will do most of the work, and tell you what to do when it needs help. If something looks scary, or you're confused, send it back to Claude and ask for more details.

Summary:

- ✓ Python 3.14.3 — already installed
- ✓ pip 25.3 — already installed (Python's package manager)
- ✓ garminconnect 0.3.6 — already installed (the Garmin data library)

You're fully set up. Whenever you're ready, we can write a script to log into your Garmin account and pull the data you want.

Claude will send you a summary upon completion, stating that all of the required elements are now installed. Onto the next step!

2.2 LOG IN ONCE

Local my-builds



Now write a small script that logs into my Garmin account with garminconnect and saves a login token on my computer, so I only sign in once. Have me type my email and password in the terminal — never in this chat.



Accept edits +

Sonnet 5 High

Allow Claude to run Claude garminconnect package source file?

Locate garminconnect package source file

This command requires approval

```
python -c "import garminconnect, inspect; print(garminconnect.__file__)"
```

Deny 1

Always allow 2

Allow once 3 Ctrl+Enter

Type / for commands

Allow once

Throughout this sequence, Claude will ask your permission to run different commands. Allow each of them.

The terminal

You'll need to paste *python "garmin_login.py"* into your terminal, so you can log into Garmin Connect once, to enable data syncing in the future. My preferred method is to access the terminal within the Claude app, which you can do by clicking the '>_' button in the top right of your session. If you're stuck, ask Claude how you can open the terminal and it'll give you instructions.

Keep your password secure

Never type your password into the Claude chat. Instead you'll enter it in the Terminal, where it goes straight to the login on your own computer and nowhere else.

```
PS C:\Users\blake\Documents\Claude Code\my-builds\Health Dashboard> python garmin_login.py
Garmin email:
Garmin password (hidden while typing):
mobile+cffi returned 429: Mobile login returned 429 – IP rate limited by Garmin
mobile+requests returned 429: Mobile login returned 429 – IP rate limited by Garmin
Signed in as Blake Froude.
Session saved to C:\Users\blake\.garminconnect – you won't need to type your password again
unless that saved session expires or you sign out.
PS C:\Users\blake\Documents\Claude Code\my-builds\Health Dashboard> []
```

Log in

Paste the command into the terminal, and follow the prompts to log in. Your Garmin password won't appear as you type, but it is there. Press 'Enter' after typing your password, and it should log in!

Getting 429 errors?

You may see a couple of 'rate limited' retry lines. That's normal, and as long as it ends with 'Signed in as...', you're in. If you have issues, try again and make sure your password is correct. You can also ask Claude for assistance if you continue to have trouble.

2.3 PULL YOUR DATA INTO ONE GROWING LOCAL DATABASE

Local my-builds



Write the sync so every run adds to one local file that builds up over time. Each run should pull my Garmin data — resting HR, HRV, sleep, body battery, stress, steps, training readiness, and all my activities — and merge it into garmin-data.json, keyed by date (and by activity ID for workouts) so re-running never makes duplicates. Keep up to the last 90 days. On the first run pull the full 90 days; after that just add what's new. Print a summary of what it added.



Accept edits +

Sonnet 5 High

One growing local database

Everything lands in a single file on your computer, and every sync tops it up. Each day and each workout has its own key, so running it twice is safe. It won't duplicate or miss activities.

To run it:

python garmin_sync.py

Make sure you're logged in first (python garmin_login.py) if you haven't already gotten past that verification step. Want me to run it now, or do you want to kick it off yourself first?

Go ahead

Checked whether a saved Garmin login session exists

A saved session exists, so this should run without asking for your password. Let's start the sync — since it's the first run, it'll pull the full 90 days and may take a few minutes.

Ran the Garmin data sync (first run, pulls full 90 days)

Bash

```
$ cd "C:\Users\blake\Documents\Claude Code\my-builds\Health Dashboard" && python garmin_sync.py
```

First run pulling the full 90 days (2026-04-11 to 2026-07-09). This can take a few minutes.

Connected as Blake Froude.

```
Fetching 2026-04-11 (1/90)...
Fetching 2026-04-12 (2/90)...
Fetching 2026-04-13 (3/90)...
Fetching 2026-04-14 (4/90)...
Fetching 2026-04-15 (5/90)...
```

Wait...

Let Claude run *python garmin_sync.py* for you.

This may take a while (5-10 minutes) to run, as it collects the past 90 days of data from Garmin.

You won't need to enter your password again, as you already logged into Garmin in Step 2.2.

2.4 WIRE IT UP AND ENABLE FUTURE SYNCs

Local my-builds



Update health-dashboard.html to read garmin-data.json and fill the metric cards, the charts and the activities list from it. Add a 'Last synced' line sourced from the data file. Wire the Sync button so clicking it runs the pull (garmin_sync.py), merges any new data into the store, and refreshes the page. Nothing runs in the background — data only updates when I press Sync.

Important: a browser can't execute a Python script or reliably read a local JSON file over file://, so this needs a small local server from the start:

- A minimal Python server (stdlib only, no new dependencies) that serves the dashboard files and exposes an endpoint the Sync button calls, which runs garmin_sync.py and returns the result.
- A double-clickable launcher appropriate for my operating system (a .bat file on Windows, a .command or .sh file on Mac/Linux) that checks whether the server is already running, starts it if not, and opens the dashboard in my default browser — so I never have to manually type a command in a terminal. Detect my OS and create the right file type, with correct permissions (e.g. executable bit set on Mac/Linux). Double-clicking that launcher (or a shortcut to it) should be the only thing I ever do to open the dashboard.

Test the full flow end-to-end before calling it done: load the dashboard with real data, click Sync, and confirm it actually merges new data and refreshes — plus confirm the launcher starts the server and doesn't spawn a duplicate if it's already running.

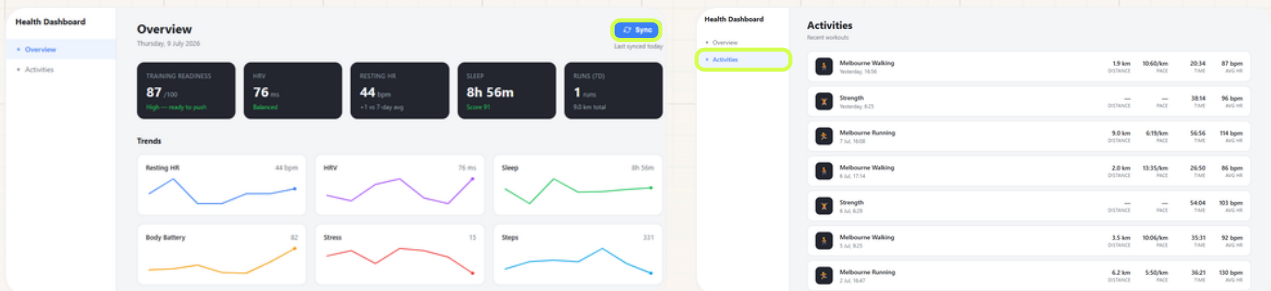


Accept edits +

Sonnet 5 High

One click to open

Claude will create a *start-dashboard.bat* file (Mac *.command* or *.sh*) in your working folder. Whenever you want to open the dashboard, just double click this file. A small terminal window opens and stays open — it's the little server that powers the *Sync* button. This launcher is the price of keeping the app local, but in the advanced module we streamline. Once the app is open, press *Sync* and the new day merges into your history.



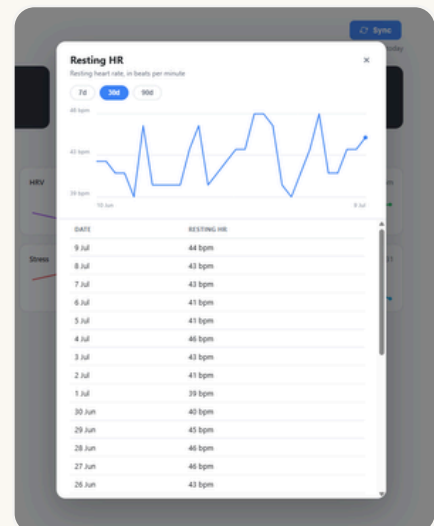
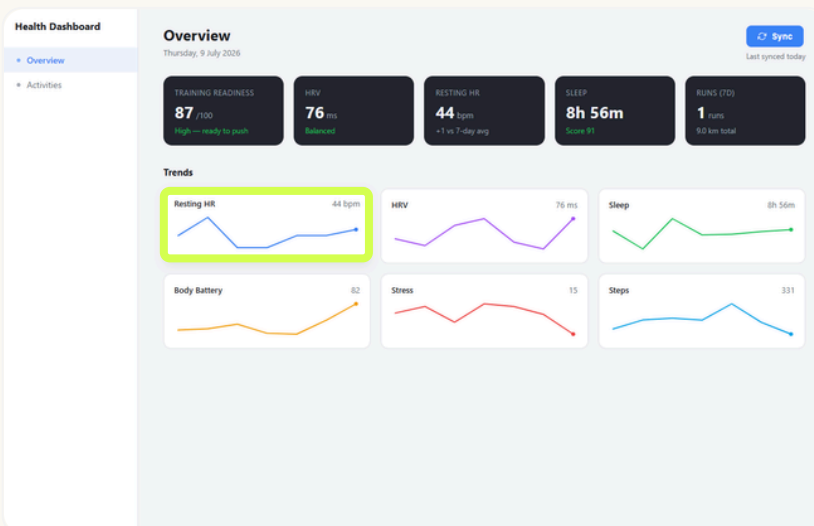
2.5 SEE ANY METRIC OVER TIME

Local my-builds

Make each card on the Overview page clickable. When I click a metric — say HRV or Sleep — open a clean popup showing that metric over time as a simple line chart, with buttons to switch between the last 7, 30 and 90 days, and the numbers in a small table underneath. Use the history in garmin-data.json. Keep it readable.

Accept edits +

Sonnet 5 High



Make it yours

The dashboard is live, so now it's time to make it yours. The possibilities are endless, so the prompts below are just a starting point.

3.1 NAME IT

Localmy-builds

Rename the dashboard from 'Health Dashboard' to [YOUR CHOICE OF NAME]. Update the sidebar title and the browser tab.

Accept edits +

Sonnet 5High

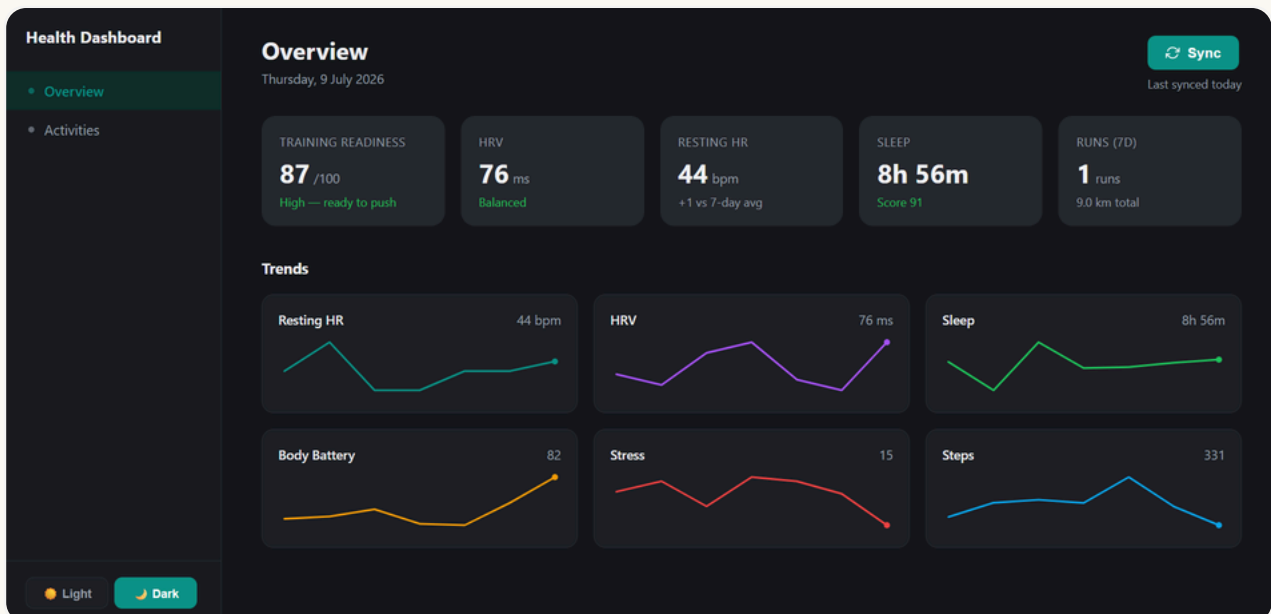
3.2 COLOUR IT

Localmy-builds

Change the accent colour to [YOUR COLOUR], and let me choose a light or dark background. Keep it clean and easy to read.

Accept edits +

Sonnet 5High

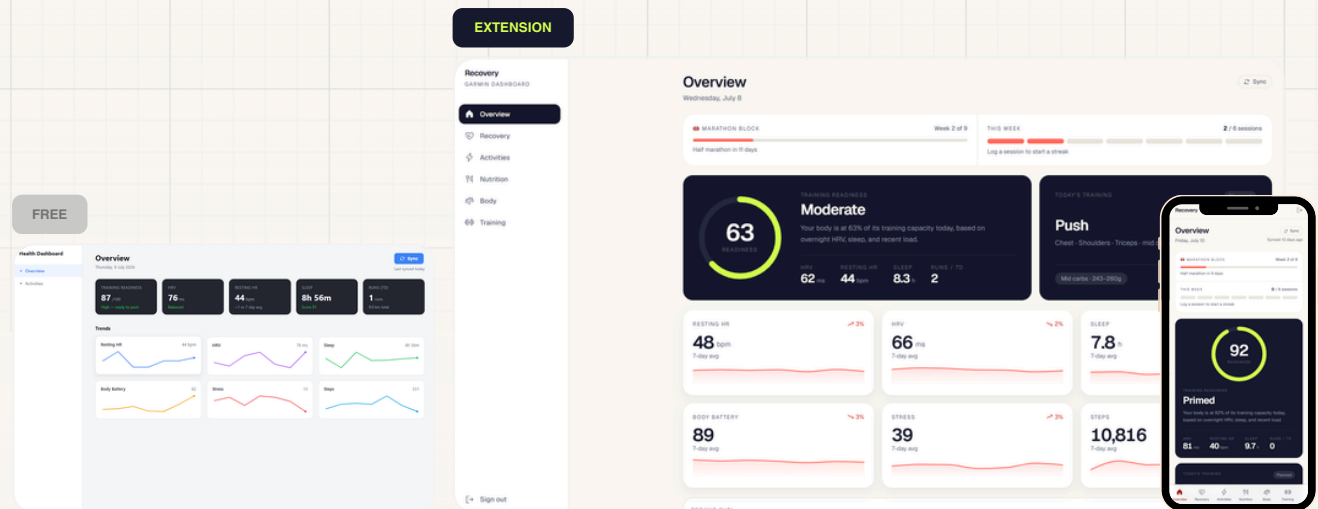


This is the whole idea

Everyone runs the same prompts, but no two dashboards come out the same. Yours has your name, your colour, and your metrics. That's build&keep — mass-produced steps, one-of-a-kind result.

You built it. Now keep it.

You've built a real, working dashboard that you own outright. Synced, local and no monthly fee. Now you have the data in one place, the possibilities are endless.



	FREE	EXTENSION
Customise it	Rename & recolour	Fully polished, unique UI
See your history	Up to 90 days	Unlimited, forever
Keep it synced	One tap	Automatic, every day
Get insights	-	Personalised recommendations
Access it	Your computer only	Anywhere, anytime
Price	Free forever	A\$19

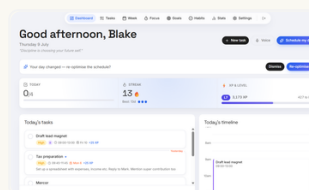
Each build = one subscription cancelled

You've just started building a tool that you can actually use, and that has the potential to replace a paid subscription to achieve similar results. The extension brings more insights, polish, access and usability.

Unlock the Extension — A\$19 one-off →

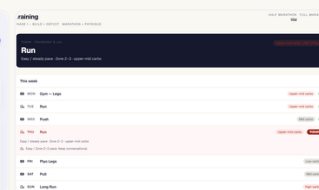
or get every build with
the lifetime bundle for A\$75

More builds you could keep



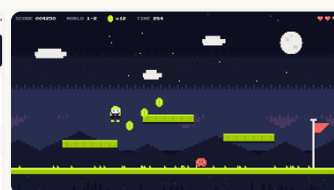
Personal Assistant

Your week at a glance, tailored to your style, with AI-driven features like automatic time-blocking



Running Coach

Turns raw data into a coach that adjusts your week, built to replace a \$15+/month app



Arcade Game

A real browser game your friends can play: scoreboard, sound, growing difficulty



Content Command Centre

A month of posts planned, drafted in your voice, with analytics on what's working

build&keep.